XRAY FOR JIRA

Complete Beginner’s Guide for QA

Concepts • Configuration • Test Design • Preconditions • Test Sets • Test Plans • Test Executions • Parameterization

Study guide

A structured reference covering the topics requested, with definitions, examples, workflows, comparison tables, and a complete end-to-end login-testing example.

Note: Xray Cloud and Xray Server/Data Center can differ in interface labels and feature availability. Use this guide to understand the concepts; confirm exact menus and parameterization behavior in the version installed in your Jira project.

Contents

- 1. What is Xray?
- 2. Major features of Xray
- 3. Configuring Xray
- 4. Important Xray definitions
- 5. Types of Tests in Xray
- 6. Why Tests matter
- 7. Creating a Test
- 8. Writing good Test Steps
- 9. Test Step vs Test
- 10. How to “call” a Test
- 11–13. Preconditions: definition, creation, association
- 14–17. Test Execution, creating it, running Tests, and key differences
- 18–22. Test Sets: definition, purpose, creation, adding to an Execution
- 23–26. Test Plans: definition, example, adding Tests, Test Plan vs Test Set
- 27. Complete relationship diagram
- 28–37. Parameterization: concepts, parameters, setup, execution, results
- 38. Complete real-world example
- 39. Important differences at a glance
- 40. Xray lifecycle and study order

1. What is Xray?

Definition: Xray is a test-management application integrated with Jira that supports the software testing lifecycle, including test design, test planning, test execution, traceability, and reporting.

It uses Jira issue/work-item types such as Test, Precondition, Test Set, Test Plan, and Test Execution to represent different testing entities.

Why use Xray?

Without a test-management tool, teams may maintain requirements, test cases, execution results, and reports in separate spreadsheets and documents. Xray connects test artifacts and execution information within Jira.

```
Jira Requirement / Story
          ↓
         Test
          ↓
   Test Execution
          ↓
     Test Run/result
          ↓
 Requirement coverage and reports
```

The central idea is traceability: a team can connect what the system should do with the tests that verify it and the results produced when those tests are executed.

2. Major Features of Xray

2.1 Test management

Create and manage manual, generic/automated, and BDD-style tests where supported by the installed version and configuration. Manual tests can contain ordered steps with actions, data, and expected results.

2.2 Test planning

Use Test Plans to define and track a testing scope, and Test Executions to record specific testing activities and their results. Test Sets can organize tests into reusable groups.

2.3 Test organization

Tests can be organized using Test Sets, test repositories/folders where available, components, versions, labels, and other Jira fields.

2.4 Requirement traceability

```
Requirement / Story
        ↓
       Test
        ↓
 Test Execution
        ↓
    Test Run
        ↓
    PASS / FAIL
```

Traceability helps answer questions such as: Which requirements have tests? Which tests failed? Which requirements lack coverage? What evidence or results are associated with an execution?

2.5 Execution tracking

Xray tracks test-run statuses such as TODO, EXECUTING, PASS, FAIL, or ABORTED, subject to the version and project configuration. Status names and workflows may be configurable.

2.6 Reporting

Reporting can include execution progress, requirement coverage, traceability, Test Plan status, test results, and historical testing information.

2.7 Automation integration

Xray can integrate with automation tools and frameworks such as Jenkins, Selenium-based suites, JUnit, TestNG, Cucumber, Robot Framework, NUnit, and REST APIs. The exact integration route depends on the framework and Xray deployment.

3. Configuring Xray

Before creating tests, Xray must be available and configured for the Jira project. The exact steps depend on whether the project uses Xray Cloud or Server/Data Center, and on the permissions granted to the user.

```
Jira Project
  ■■■ Xray issue types
  ■     ■■■ Test
  ■     ■■■ Precondition
  ■     ■■■ Test Set
  ■     ■■■ Test Plan
  ■     ■■■ Test Execution
  ■■■ Test types
  ■■■ Test-step fields
  ■■■ Test statuses / workflows
  ■■■ Requirement coverage configuration
```

Typical configuration checklist

- Confirm that Xray is installed/licensed and enabled for the Jira site.
- Confirm that the relevant Xray issue types are available in the project.
- Check project permissions and issue-type/workflow configuration.
- Identify which Jira issue types are considered testable/coverable requirements, such as Stories or Epics.
- Review the test types and manual-step fields enabled in the project.
- Check statuses, execution workflow, and reporting/coverage settings.

Typical navigation may include Project Settings, Xray or Apps settings, and Jira Administration → Manage Apps → Xray. These are illustrative paths, not guaranteed menu names for every deployment.

4. Important Xray Definitions

4.1 Test

Definition: A Test is an Xray work item that defines how a requirement or other testable item will be validated. A manual Test usually contains ordered steps with an action, test data, and expected result.

4.2 Test Step

A Test Step is one action or check within a manual Test. A Test contains the overall scenario; its steps explain how to perform and verify it.

4.3 Precondition

A Precondition describes a condition that must be satisfied before a Test can be executed, such as having an active user account.

4.4 Test Set

A Test Set is a logical collection of Tests used mainly to organize related Tests. It does not primarily represent execution results.

4.5 Test Plan

A Test Plan defines and tracks a testing scope, such as a release, sprint, version, or testing objective. It can group Tests and consolidate results from related Test Executions.

4.6 Test Execution

A Test Execution represents a particular testing activity and contains Tests to execute, with associated Test Runs and results. Examples include a regression run in QA or a smoke test on a specific build.

4.7 Test Run

A Test Run is the execution instance of a Test within a Test Execution. It carries the run status and, where supported, step results, comments, evidence, and execution details. It is not itself a Jira issue.

4.8 Requirement / Story

A Requirement or Story describes what the system should do. In Jira, this is often a Story, Bug, or another issue type configured as coverable by tests.

4.9 Parameter

A parameter is a variable value used by a test so that the same test logic can be reused with different data. Parameterization features and syntax depend on the Xray version and test type.

5. Types of Tests in Xray

Available test types depend on the Xray version, project configuration, and integrations. Common concepts include:

5.1 Manual Test

A human tester follows predefined steps and records whether the actual behavior matches the expected results.

```
Action: Open the login page.
Expected Result: The login page is displayed.
```

5.2 Generic Test

A Generic Test can represent an automated test where the implementation lives outside the Xray Test definition. Xray tracks the test identity and imported/associated result.

```
Test: Verify Login API
Automation reference: LoginTest.verifyValidLogin()
```

5.3 Cucumber / BDD Test

Behavior-Driven Development (BDD) tests can be expressed in Gherkin using Given, When, and Then.

```
Given the user is on the login page
When the user enters valid credentials
Then the dashboard should be displayed
```

Do not assume every project has all test types enabled. Check the test-type selector and project setup in your installed version.

6. Why a Test is Important

A Test makes verification repeatable and traceable. For example, a Story says a user should be able to log in; a Test defines the steps to verify login; an execution records what happened when the Test was run.

```
STORY-100: User should be able to log in
                ↓
TEST-101: Verify valid login
                ↓
TE-010: Regression Testing - Release 2.0
                ↓
Test Run: PASS
```

7. Creating a Test in Xray

A typical workflow is:

```
Create issue / work item
        ↓
Select project
        ↓
Issue type = Test
        ↓
Enter summary and relevant fields
        ↓
Choose test type (for example, Manual)
        ↓
Create the Test
        ↓
Open the Test and define its steps
```

Example summary: Verify user login with valid credentials.

Test type: Manual.

Depending on the Xray version, the steps editor may be shown in the issue screen or in a dedicated Test details section.

8. How to Write Good Test Steps

Manual Test steps commonly use these fields:

Field	Meaning	Example
Action	What the tester should do.	Enter a valid registered email address in the Email field.
Data	The input data required for the step.	user@example.com
Expected Result	What should happen after the action.	The email is accepted and displayed in the field.
Attachment	Supporting evidence, where available.	Screenshot or relevant file.

In some Xray configurations, only Action is mandatory by default; fields and requirements may be customized.

Bad vs better test step

Bad: “Login.” This is too vague.

Better: “Enter a valid registered email address in the Email field.”

Expected result: “The email address is accepted and displayed in the field.”

Example: Verify valid login

Step	Action	Data	Expected Result
1	Open the login page.	Application URL	Login page is displayed.
2	Enter a valid username.	user@example.com	Username is accepted.
3	Enter a valid password.	Password stored securely in test data	Password field accepts the input.
4	Click Login.	N/A	User is redirected to the Dashboard.

Best practice: keep each step to one clear action or check. Do not combine opening the application, entering credentials, clicking Login, and checking the dashboard into one enormous step. Split it into separately verifiable actions.

9. Test Step vs Test

Test	Test Step
The complete test case.	One action/check inside the test.
Example: TEST-101 — Verify valid login.	Example: Step 1 — Open application.
Contains the scenario and its ordered steps.	Provides the detailed action and expected behavior.

```
TEST-101: Verify valid login
  ■■■ Step 1: Open application
  ■■■ Step 2: Enter username
  ■■■ Step 3: Enter password
  ■■■ Step 4: Click Login and verify Dashboard
```

10. How to “Call” a Test

In ordinary Xray manual testing, “call a Test” usually means selecting or adding the Test to a Test Execution. It does not normally mean calling it like a Java method. Automation integrations can have their own test references or identifiers.

```
Existing Tests
  TEST-101
  TEST-102
  TEST-103
       ↓ add/select
Test Execution
  ■■■ TEST-101
  ■■■ TEST-102
  ■■■ TEST-103
```

Once included in an execution, each Test has a Test Run in that execution. The tester or automation integration records the result there.

11. What is a Precondition?

Definition: A Precondition defines what must be true or already in place before a Test can be executed.

Example: For “Verify successful login,” a suitable Precondition might be “A registered, active user account exists.”

```
PRE-001: Active user account exists
                  ↓
TEST-101: Verify successful login
```

Preconditions can be reused by multiple Tests, avoiding repeated setup text.

12. Creating a Precondition

A typical workflow is:

```
Create issue / work item
        ↓
Select issue type = Precondition
        ↓
Enter a clear summary
        ↓
Describe the required condition
        ↓
Create and save
```

Example summary: Active user account exists.

Condition description: A valid user account exists in the test environment and its account status is Active.

13. Associating a Precondition with a Test

Open the Test and find its Precondition area/field. Use the relevant add/link/select action to associate the existing Precondition. The exact control differs by Xray version.

```
PRE-001: Active user account exists
          ■■■ TEST-101: Verify valid login
          ■■■ TEST-102: Verify logout
          ■■■ TEST-103: Verify session timeout
```

One Precondition can be associated with multiple Tests. Association records the setup requirement; it does not automatically create the account or guarantee that the condition is true. The tester or test environment must ensure it is satisfied.

14. What is Test Execution?

Definition: A Test Execution is an Xray work item representing a specific testing activity. It contains the Tests being executed and their Test Runs/results.

A useful way to remember it:

```
Test           = What should I test?
Test Execution = In which testing activity am I running it?
Test Run       = What happened when this Test was run in that Execution?
```

Example: Test = “Verify Login”; Test Execution = “Regression Testing — Release 2.0, QA environment, Chrome.”

15. Creating a Test Execution

Typical workflow:

```
Create issue / work item
        ↓
Select issue type = Test Execution
        ↓
Enter summary and relevant execution details
        ↓
Create and save
        ↓
Add/select Tests for execution
```

Example summary: “Regression Test Execution — Release 2.0.” Relevant fields may include environment, version, build, assignee, or other project-specific fields.

```
TEST EXECUTION: Regression Run
  ■■■ TEST-101
  ■■■ TEST-102
```

```
■■■ TEST-103
■■■ TEST-104
```

16. Running a Test

A typical manual execution workflow is:

```
Open Test Execution
        ↓
Open/select a Test Run
        ↓
Perform the test steps
        ↓
Record step results (where supported)
        ↓
Add comments and evidence if useful
        ↓
Set the overall result/status
```

Example successful run:

```
Step 1 → PASS
Step 2 → PASS
Step 3 → PASS
Step 4 → PASS
Overall Test Run → PASS
```

Example failed run:

```
Step 1 → PASS
Step 2 → PASS
Step 3 → FAIL
Step 4 → not executed / as appropriate
Overall Test Run → FAIL
```

Record what actually happened and attach useful evidence according to your team's process. If a test fails, document the observed behavior and raise/link a defect when appropriate. Do not mark a test PASS just because the steps were attempted.

17. Test vs Test Run vs Test Execution

Entity	Meaning	Example
Test	Definition of what should be tested.	TEST-101: Verify Login
Test Step	Individual action/check within a Test.	Enter username.
Test Execution	Specific testing activity containing Tests.	Regression Run — Release 2.0
Test Run	Instance/result of one Test within an Execution.	TEST-101 run on Chrome, result PASS.

```
TEST-101: Verify Login
        ↓ added to
TE-010: Regression Run
        ↓ creates/contains
Test Run for TEST-101
        ↓ result
PASS / FAIL / other status
```

This distinction is fundamental: the Test defines the scenario, the Test Execution represents the testing activity, and the Test Run stores the result for that Test in that activity.

18. What is a Test Set?

Definition: A Test Set is a logical, ordered collection of Tests grouped mainly for organization. It is not primarily a record of execution results.

```
TEST SET: Login Regression Tests
  ■■■ TEST-101: Valid Login
  ■■■ TEST-102: Invalid Password
  ■■■ TEST-103: Invalid Username
  ■■■ TEST-104: Locked Account
  ■■■ TEST-105: Logout
```

19. Why Use Test Sets?

Suppose a project has hundreds of Tests and repeatedly needs to select Login, Payment, Registration, and Search tests. Test Sets let the team maintain reusable logical groupings instead of searching for each Test individually every time.

A Test Set helps with organization and selection. The actual run status and result belong to Test Runs within Test Executions.

20. Creating a Test Set

Typical workflow:

```
Create issue / work item
        ↓
Select issue type = Test Set
        ↓
Enter summary (e.g., Login Regression Tests)
        ↓
Add/select existing Tests
        ↓
Save
```

```
TEST SET: Login Regression
  ■■■ TEST-101
  ■■■ TEST-102
  ■■■ TEST-103
```

21. Adding a Test Set to an Execution

The precise UI varies by Xray version. Conceptually, use the Test Set to select its Tests, then add those Tests to a Test Execution. Some versions/workflows offer a direct action for adding Tests from a Test Set.

```
TEST SET: Login Regression
        ↓ select/add its Tests
TEST EXECUTION: Regression Run
  ■■■ TEST-101
  ■■■ TEST-102
  ■■■ TEST-103
```

Important: the Test Set organizes the Tests; the Test Execution records the testing activity and results. Adding a Test Set does not itself mean that the Tests have been executed.

22. Test Set vs Test Execution

Test Set	Test Execution
Organizes Tests logically.	Represents a particular testing activity.
Reusable grouping of Tests.	Contains Tests to execute and their Test Runs.
Does not primarily store execution results.	Stores/tracks execution results through Test Runs.
Example: Login Regression Tests.	Example: Regression Run — Oct 27.

23. What is a Test Plan?

Definition: A Test Plan is an Xray work item used to plan and track a testing effort for a particular scope, such as a release, sprint, version, or testing objective. It can group Tests and consolidate results from related Test Executions.

Remember: Test Plan answers, “What testing are we planning and tracking?”

```
TEST PLAN: Release 2.0 Testing
  ■■■ Login tests
  ■■■ Payment tests
  ■■■ Registration tests
  ■■■ Search tests
```

24. Test Plan Example

Imagine a Veeva-like application release contains Login, Document Management, Approval Workflow, and User Management requirements. Create a Test Plan such as “Release 5.0 QA Plan” to define and track the tests needed for that release.

```
TEST PLAN: Release 5.0 QA Plan
  ■■■ Tests for Login
  ■■■ Tests for Document Management
  ■■■ Tests for Approval Workflow
  ■■■ Tests for User Management
```

25. Adding Tests to a Test Plan and Using It

Typical workflow:

```
Create Test Plan
        ↓
Add/select the Tests in scope
        ↓
Create related Test Executions as needed
        ↓
Execute Tests in each Execution
        ↓
Review consolidated progress/results
```

```
TEST PLAN: Release 5.0 QA Plan
          ■■■■■■■■■■■■■■■■■■
          ↓                 ↓
Execution 1            Execution 2
Smoke Testing          Regression Testing
          ↓                 ↓
Test Runs/results      Test Runs/results
          ■■■■■■■■■■■■■■■■■■
                    ↓
       Review plan progress/results
```

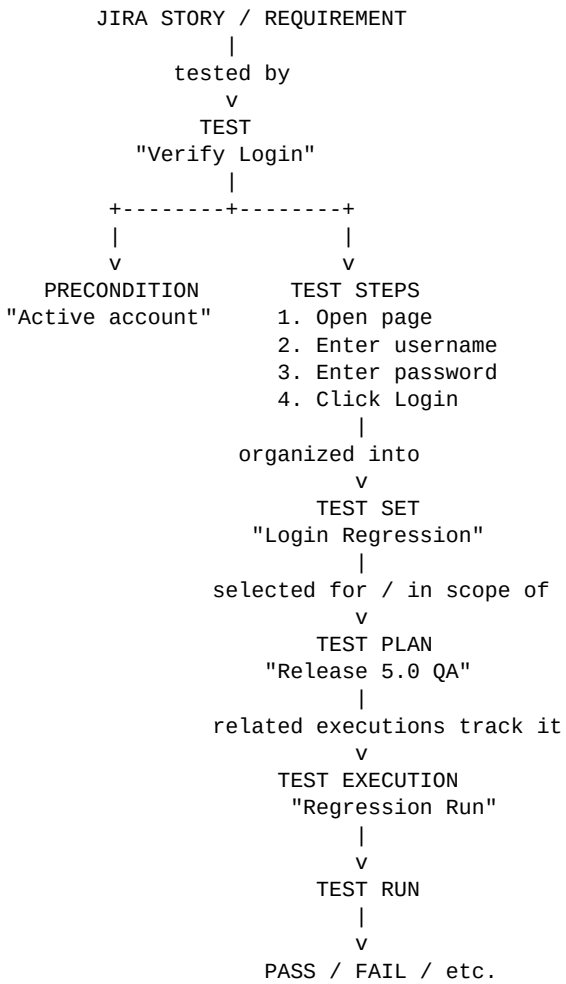
Exact workflows for adding Tests, creating executions from a plan, and displaying consolidated results can differ between Xray versions. The core concept remains planning and tracking a testing scope.

26. Test Plan vs Test Set

Test Set	Test Plan
Answers: Which Tests belong together?	Answers: What testing effort are we planning/tracking?
Main purpose is organization.	Main purpose is planning and tracking a scope.
Example: Payment Tests.	Example: Release 5.0 Testing.
Can be reused as a logical group.	Can track Tests and related execution results for a scope.

Do not treat Test Set and Test Plan as interchangeable. A Test Set is an organizational collection; a Test Plan represents a testing scope and supports planning/tracking.

27. Complete Xray Relationship



This is a conceptual relationship map. A real Jira/Xray project may allow several relationships and workflows beyond this simplified diagram.

28. Parameterization in Xray

Parameterization lets test logic be reused while input values vary. It is especially useful when the same scenario must be tested with multiple users, roles, browsers, environments, quantities, or other data.

Example: instead of maintaining several nearly identical login tests for user1, user2, and user3, a parameterized test can use variable values—if the Xray feature/test type being used supports that workflow.

```
Test logic:
  username = ${username}
  password = ${password}

Data combination 1:
  username = user1
  password = pass1

Data combination 2:
  username = user2
  password = pass2

Data combination 3:
  username = user3
  password = pass3
```

Important qualification: do not assume that every Xray manual Test supports a universal built-in parameter table or that every parameterization approach creates separate Test Runs automatically. Parameterization behavior depends on Xray Cloud vs Server/Data Center, the test type, and the particular data-driven or automation integration. Verify the feature available in your installation.

29. Why Parameterization?

Without reuse, teams may duplicate similar test cases:

```
TEST-101 → user1
TEST-102 → user2
TEST-103 → user3
```

With a parameterized approach, the same logical test can be applied to different data combinations:

```
ONE TEST LOGIC
  ■■■ Data set 1
  ■■■ Data set 2
  ■■■ Data set 3
  ■■■ Data set 4
```

Benefits include reusability, less duplication, easier maintenance, wider data coverage, and clearer comparison of behavior across data values. Parameterization does not replace good test design: each data combination must still have a clear purpose and expected result.

30. What is a Parameter?

A parameter is a variable placeholder whose value is supplied for a particular test-data combination.

```
${username}
${password}
${role}
${browser}
${environment}
```

Parameter	Example value
username	user1
password	A test-only password stored securely
role	Reviewer
browser	Chrome
environment	QA

31. Parameter Data Categories

There is not one universal list of primitive “Xray parameter types” that applies to every Xray version and feature. For general QA thinking, test data can fall into categories such as:

Data category	Example
String/text	username = "jaideep"
Number	quantity = 10
Boolean	isActive = true
Date	expiryDate = 2026-12-31
Combinations of test data	username, password, role, browser, environment

These are general data categories, not a claim that Xray exposes each as a native parameter type in every configuration. The exact parameter format, data source, and supported types depend on the feature or integration used.

32. Creating Parameters

Conceptually, a parameterized workflow requires you to define variable names, supply data values, and reference those variables in the test or automation implementation.

```
Open the Test / test-data configuration
        ↓
Define variables (e.g., username, password)
        ↓
Provide data values or data combinations
        ↓
Reference the variables in the steps or automation
        ↓
Save and verify the supported execution workflow
```

The actual UI is version- and feature-dependent. Do not assume there is a single menu called “Parameters” in every Xray installation.

33. Adding Parameters to a Test

Example logical Test: TEST-101 — Verify Login.

Parameters: username and password.

```
Step 1: Open Login Page
Expected: Login page is displayed.

Step 2: Enter ${username}
Expected: Username is accepted.

Step 3: Enter ${password}
Expected: Password field accepts the input.

Step 4: Click Login
Expected: Application responds according to the supplied credentials.
```

For positive login data, the expected result may be a Dashboard. For negative data, the expected result should instead describe the appropriate error or rejection. Do not use one expected result for data combinations that are meant to produce different behavior.

34. Executing Parameterized Tests

Suppose a test uses three data combinations:

Data set	Username	Password	Expected behavior
1	user1	Valid test credential	Login succeeds
2	user2	Valid test credential	Login succeeds
3	locked_user	Valid test credential	Login is rejected because the account is locked

A data-driven execution process supplies each combination to the test logic. How the combinations are scheduled and how results are represented depends on the selected Xray parameterization/automation method. Make sure the report can identify which data set was used for each result.

35. Calling a Parameterized Test

For manual testing, select the Test for execution and supply/use the appropriate values through the supported workflow. For automated testing, the test runner or data-driven framework may pass the values to the implementation, and Xray may receive the results through the configured integration.

```
Test Execution / supported runner
            ↓
Parameterized Test
            ↓
Select or supply data combination
            ↓
Execute test logic
            ↓
Record/import result
```

Do not confuse this with calling a Java method. The details depend on whether the Test is manual, generic/automated, or BDD-driven.

36. Viewing Parameterized Results

A useful result view should make clear which data combination passed or failed. For example:

Test	Username/data set	Result
Login Test	user1	PASS
Login Test	user2	PASS
Login Test	user3	FAIL

This is more informative than seeing only “Login Test = FAIL,” because it identifies the failing data combination. Depending on the implementation, the per-data results may appear as separate executions, iterations, test-run details, or automation-report entries; confirm how your setup represents them.

37. Parameterization: Practical Cautions

- Use synthetic or approved test credentials; do not place real user passwords in documentation or reports.
- Define expected results for each meaningful data combination.
- Include boundary, invalid, role-based, and state-dependent data when appropriate.
- Keep the source of data and the relationship between a result and its input values clear.

- Confirm how your Xray version and chosen integration store or display iteration-level results.
- Avoid duplicating dozens of tests when the behavior is genuinely the same, but do not force unrelated scenarios into one parameterized test.

38. One Complete Real-World Example

Requirement

```
STORY-100
As a registered user, I should be able to log in
using valid credentials.
```

Precondition

```
PRE-001
A valid, active registered user exists in the test environment.
```

Test

```
TEST-101
Verify successful login
```

Steps

Step	Action	Expected Result
1	Open the application.	Login page is displayed.
2	Enter ${username}.	Username input is accepted.
3	Enter ${password}.	Password input is accepted.
4	Click Login.	User is authenticated and redirected to the Dashboard.

Test Set

```
TS-001: Login Regression
  ■■■ TEST-101
  ■■■ TEST-102
  ■■■ TEST-103
  ■■■ TEST-104
```

Test Plan

```
TP-001: Release 5.0 QA Plan
Scope: Tests required for Release 5.0.
```

Test Execution

```
TE-001: Release 5.0 Regression - Chrome
  ■■■ TEST-101
  ■■■ TEST-102
  ■■■ TEST-103
  ■■■ TEST-104
```

Example Test Run outcomes

Test	Outcome
TEST-101	PASS
TEST-102	PASS
TEST-103	FAIL
TEST-104	PASS

End-to-end relationship

```
STORY-100
    ↓ tested by
TEST-101
    ■■■ PRE-001 (setup condition)
    ■■■ ordered Test Steps
            ↓ organized into
        TS-001 (Test Set)
            ↓ included in testing scope of
        TP-001 (Test Plan)
            ↓ executed in
        TE-001 (Test Execution)
            ↓ produces
        Test Runs and results
            ↓
        PASS / FAIL / other status
```

This example is simplified for learning. In a real project, the exact links, field names, test data configuration, and execution workflow depend on the Jira/Xray setup.

39. Important Differences at a Glance

Concept	One-line meaning
Xray	Test-management application integrated with Jira.
Requirement / Story	What the system should do.
Test	How the requirement or behavior will be verified.
Test Step	One action/check inside a Test.
Precondition	What must be true before the Test runs.
Test Set	A logical group of Tests for organization.
Test Plan	The testing scope/effort being planned and tracked.
Test Execution	A specific testing activity containing Tests and their runs.
Test Run	The instance and result of one Test within an Execution.
Parameter	A variable test-data value used to reuse test logic.

The four distinctions to memorize

```
Test Set      = ORGANIZATION
Test Plan     = PLANNING + TRACKING
Test Execution= EXECUTION ACTIVITY
Test Run      = RESULT OF A TEST IN THAT EXECUTION
```

40. Xray Lifecycle for QA Work

If asked to explain your Xray testing process, describe the workflow in order rather than listing menu names randomly.

```
1. Review the Requirement / Story
                ↓
2. Analyze the expected behavior
                ↓
3. Create the Test
                ↓
4. Write clear Test Steps and expected results
                ↓
5. Create and associate Preconditions
                ↓
6. Organize Tests into Test Sets (when useful)
                ↓
7. Create a Test Plan for the scope
                ↓
8. Add/select Tests in the plan
                ↓
9. Create a Test Execution
                ↓
10. Add/select Tests to execute
                ↓
11. Execute Test Runs
                ↓
12. Record PASS / FAIL / other status
                ↓
13. Attach evidence and comments
                ↓
14. Log or link defects for failures
                ↓
15. Review requirement coverage and results
                ↓
16. Report testing status
```

Recommended learning order

- Jira/Xray configuration and issue types.
- Requirements and traceability.
- Create a Manual Test and write professional steps.
- Create and associate Preconditions.
- Create a Test Set and add Tests.
- Create a Test Plan and understand its scope.
- Create a Test Execution, run Tests, and record evidence/results.
- Understand Test Runs, statuses, defects, coverage, and reporting.
- Learn parameterization using the specific Xray version/feature used by your project.
- Then learn automation integration and REST API if needed.

Quick revision: interview-ready explanations

Question	Short answer
What is Xray?	A Jira-integrated test-management tool for test design, planning, execution, traceability, and reporting.
What is a Test?	A definition of how a behavior or requirement will be verified.
What is a Precondition?	A condition that must be satisfied before a Test can be executed.
What is a Test Set?	A logical group of Tests for organization.
What is a Test Plan?	A scope used to plan and track testing and related execution results.
What is a Test Execution?	A particular testing activity that contains Tests and their Test Runs.
What is a Test Run?	The execution instance and result of a Test within a Test Execution.
What is parameterization?	Reusing test logic with different input data values, using a supported feature or integration.
How do you run a Test?	Add/select it in a Test Execution, execute the steps or automation, and record/import the result.

Reference notes

This guide preserves the concepts and examples from the earlier explanation while clarifying that UI labels and some features vary by Xray version. For exact current instructions, consult the official Xray documentation for your deployment:

Xray overview and documentation: https://docs.getxray.app/display/XRAYCLOUD/Overview

Project organization: https://docs.getxray.app/space/XRAYCLOUD/44566397/Project%20Organization

Test Sets vs Test Plans: https://docs.getxray.app/space/XRAY/301474179/Test%20Sets%20vs%20Test%20Plans

The guide is for learning and interview preparation. Your project's administrator and installed Xray version determine the exact configuration, menu paths, fields, and supported parameterization workflow.